DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Name:M.RAMAKRISHNA ID: 2400031163

#SKILL - 5 Spring Autowiring Demo using @Autowired

Prerequisites:

 General Idea on Spring Framework

 Modules of Spring Framework

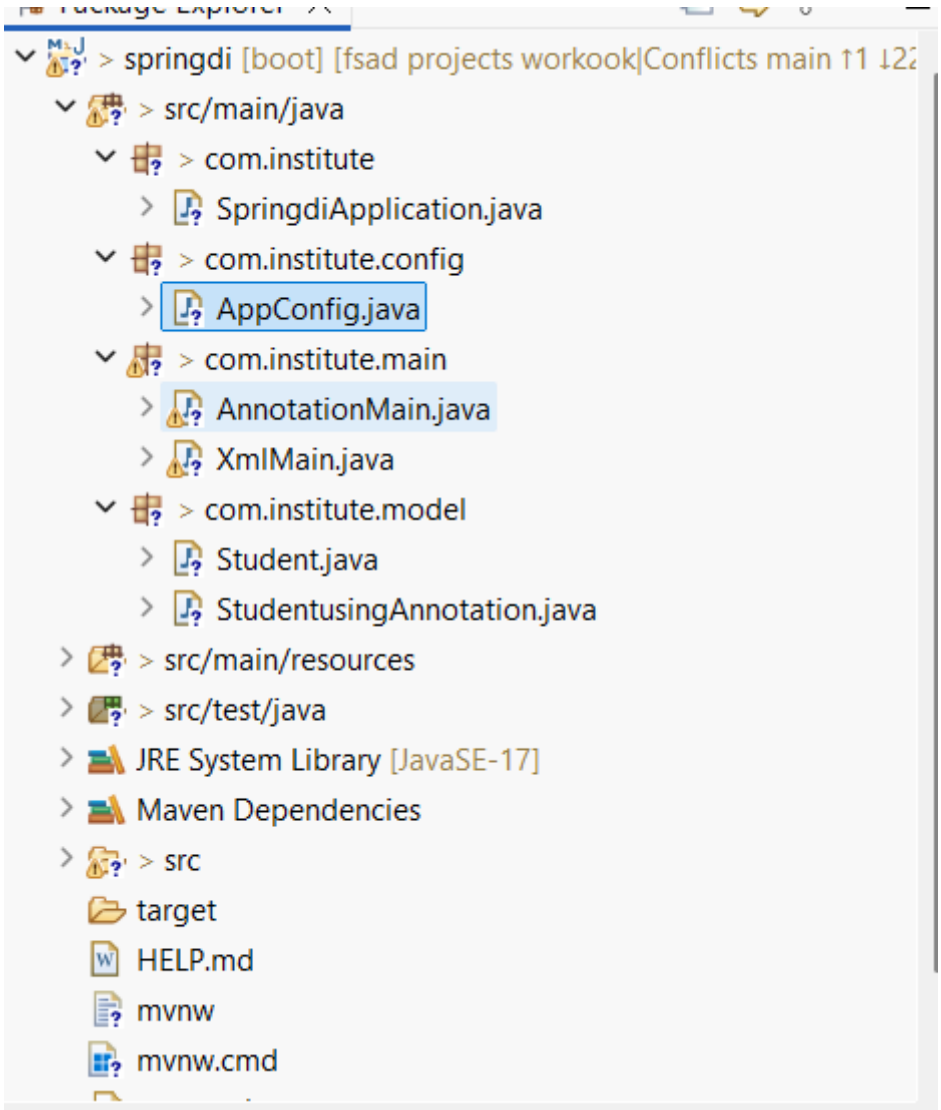
 Understanding of Autowiring concepts

A college application maintains student information along with their certification details. The
Student object contains a Certification object, and instead of creating this Certification object
manually, Spring should automatically provide it using the @Autowired annotation. When the
required classes are marked with @Component, Spring creates their objects and manages them inside
the IoC container. Using @Autowired, the Certification object is supplied to the Student object by
Spring without using the new keyword, allowing both objects to be linked through annotation-based
configuration.

Tasks:

1. Create a Certification class with fields such as id, name, and dateOfCompletion.
2. Create a Student class with fields such as id, name, gender, and a Certification object.
3. Annotate both classes with @Component to enable Spring detection.
4. Use @Autowired on a field, constructor, or setter inside the Student class to inject the
Certification object.
5. Configure component scanning using either:
a. An XML configuration file
b. A Java configuration class with @Configuration and @ComponentScan
6. Load the Spring IoC container using ApplicationContext.
7. Retrieve the Student bean and print all details, including the injected Certification object.
8. Push the Spring project to GitHub.

STRUCTURE:



Certification.java

```
package com.klu.model;

import org.springframework.stereotype.Component;

@Component
public class Certification {

    private int id = 501;
    private String name = "Spring Framework Certification";
    private String dateOfCompletion = "10-02-2026";

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDateOfCompletion() { return dateOfCompletion; }
```

```
    @Override
    public String toString() {
        return id + " " + name + " " + dateOfCompletion;
    }
}
```

Hibernate.cfg.xml

```
package com.klu.model;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Student {

    private int id = 101;
    private String name = "Sai Kiran";
    private String gender = "Male";

    private Certification certification;

    @Autowired
    public Student(Certification certification) {
        this.certification = certification;
    }

    @Override
    public String toString() {
        return id + " " + name + " " + gender + " | Certification: " + certification;
    }
}
```

Student.java

```
package com.klu.model;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Student {

    private int id = 101;
    private String name = "Sai Kiran";
    private String gender = "Male";

    private Certification certification;
```

```
    @Autowired
    public Student(Certification certification) {
        this.certification = certification;
    }

    @Override
    public String toString() {
        return id + " " + name + " " + gender + " | Certification: " + certification;
    }
}
```

MainApp.java

```
package com.klu.main;

import com.klu.config.AppConfig;
import com.klu.model.Student;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {

    public static void main(String[] args) {

        ApplicationContext alphaContext =
                new AnnotationConfigApplicationContext(AppConfig.class);

        Student betaStudent = alphaContext.getBean(Student.class);

        System.out.println(betaStudent);
    }
}
```

AppConfig.java

```
package com.klu.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.klu")
public class AppConfig {
}
```

AnnotationMainApp.java

```
package com.klu.annomain;

import com.klu.config.AppConfig;
import com.klu.model.Student;
import org.springframework.context.ApplicationContext;
```

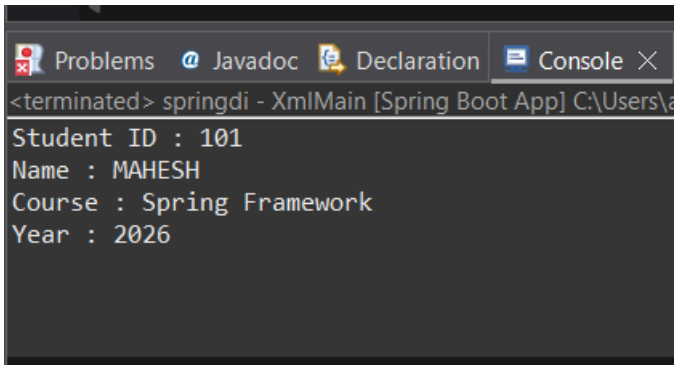
```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class AnnotationMainApp {

    public static void main(String[] args) {

        ApplicationContext deltaContext =
                new AnnotationConfigApplicationContext(AppConfig.class);

        Student epsilonStudent = deltaContext.getBean(Student.class);

        System.out.println(epsilonStudent);
    }
}
```

pom.Xml

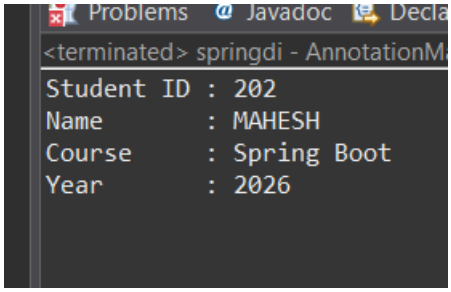
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.klu</groupId>
    <artifactId>SpringAutowireSkill5</artifactId>
    <version>1.0</version>

    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>5.3.30</version>
        </dependency>
    </dependencies>

</project>
```

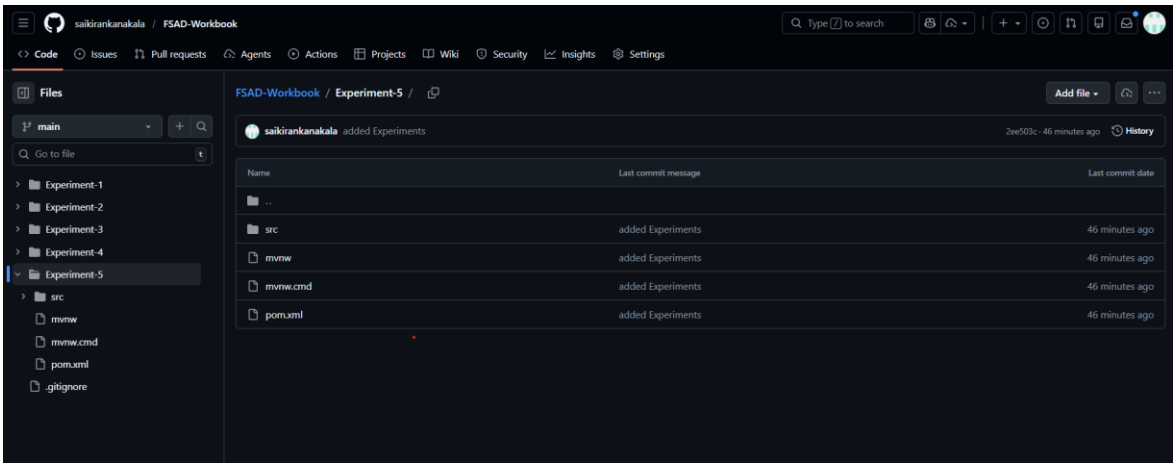
Outputs:

XmlMain.java:



AnnotationMain.java:





VIVA QUESTIONS:

1. What is autowiring in Spring?
2. What is the purpose of the @Autowired annotation?
3. How does Spring choose which bean to inject?
4. What is component scanning in Spring?
5. What happens if more than one bean matches the dependency type?

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation

Date of the Session: _/_/_ Time of The Session: _____to_____